Take-Home Assignment

Production-Ready Microservice & CI/CD

Senior Software Developer (Node.js)

Role	Senior Software Developer (Node.js)
Expected effort	About 4–6 hours
What you'll build	A lightweight Node.js REST API for task management, a React frontend that uses it, a production-grade Docker image, AWS infrastructure defined in code, and a CI/CD pipeline that tests and deploys on every push to `main`
What you'll submit	A public GitHub or GitLab repository, a live URL, and a README
AI tools	Encouraged. You must be able to explain every line you submit.

Overview

You'll build a lightweight, RESTful Task Management API in Node.js and take it all the way to production: a React single-page app that consumes it, a production-grade Docker image, AWS infrastructure provisioned as code, and a CI/CD pipeline that tests and deploys the service automatically.

The feature set is deliberately small, so you can spend your time doing it well end to end. The assignment has four steps:

Step	What you deliver
Step 1 – Backend API	Node.js REST service with three endpoints, request logging, validation, and Jest tests (≥ 80% coverage)
Step 2 – Frontend	React single-page app that lists and creates tasks and shows API health
Step 3 – DevOps	Dockerfile, AWS infrastructure in Terraform or CDK, and a CI/CD pipeline
Step 4 – Documentation	README with setup instructions and a System Architecture section

Using AI tools

- You're encouraged to use AI tools (Claude, GitHub Copilot, ChatGPT, etc.) as much as you like.
- You own everything you submit. In the technical interview you'll be asked to **explain and defend architectural decisions, specific lines of code, and/or configuration files**, including anything an AI tool generated.
- Your README must briefly say where you used AI and describe one instance where you had to correct or refine its output (see Step 4).

Ground rules

- Put everything (API, frontend, Dockerfile, infrastructure code, pipeline config) in **one public GitHub or GitLab repository**.
- All AWS infrastructure must be defined in code with **Terraform or AWS CDK**. Resources created by hand in the AWS Console are not accepted.
- Don't commit secrets such as AWS keys, SSH keys, or `.env` files. Use your CI provider's secrets store.
- If a requirement is unclear, make a reasonable assumption, note it in your README, and keep going.
- If you can't finish everything, list what's missing in your README and explain how you would complete it.

Step 1 – Backend API (Node.js)

Build a clean RESTful service that follows standard HTTP methods, status codes, and asynchronous patterns.

Requirements

Area	Requirement
Framework	Express, Fastify, or NestJS.
Data storage	Either a real database (MongoDB or PostgreSQL) or an in-memory/simulated store (e.g., an in-memory array or a JSON file). If you use an in-memory store, its data-access functions must still be asynchronous (return Promises / use `async`/`await`), the way a real database driver would be.
Middleware	Implement a **custom request-logging middleware** that logs every incoming request (e.g., method, path, status code, and response time), plus **basic request validation**.

Area	Requirement
Validation errors	Reject invalid requests with `400 Bad Request` and a clear error message.
Testing	Jest, with **at least 80% code coverage**. Unit tests must mock the data-access layer and focus on business logic.

Endpoints

Endpoint	Purpose	Expected behaviour
`GET /health`	Report service health for load balancers and monitoring	`200` with the service status and uptime, e.g. `{ "status": "ok", "uptime": 1234 }`
`GET /tasks`	List all tasks	`200` with a JSON array of tasks (`[]` when there are none)
`POST /tasks`	Create a task	The body must include `title`, a non-empty string. Success: `201` with the created task. Invalid body: `400` with an error message.

Suggested task shape (you may add fields):

```
{
  "id": "a1b2c3",
  "title": "Write the README",
  "createdAt": "2026-11-02T09:30:00Z"
}
```

> **Done when**
>
> - All three endpoints behave as described, including the `400` case.
> - `npm test` runs the suite, and backend coverage is at or above 80%.
> - Every request produces a log line from your middleware.

Step 2 – Frontend (React)

Build a clean, responsive single-page web application that consumes your API endpoints.

Requirements

Area	Requirement
Framework / build tool	React, built with Vite or Next.js (Create React App is also accepted).
Task list	Fetch and display all tasks from `GET /tasks`.
Create-task form	Add a task via `POST /tasks`, with client-side validation (the title is required and can't be empty).
UI states	Handle loading, empty (no tasks yet), and error states gracefully, e.g., show an alert when an API call fails.
Health indicator	Display a visual indicator in the UI of the API's health, based on `GET /health` (e.g., a green/red badge).
Code structure	Separate components cleanly. Put API calls in custom hooks or a state-management layer. Use standard styling (CSS Modules, Tailwind, or styled-components).
Testing	The 80% coverage requirement applies to the backend. Frontend tests are welcome but not required.

> **Done when**
>
> - A user can view tasks, add a task, and see it appear in the list.
> - Invalid input is caught in the form before it reaches the API.
> - When the API is down, the app shows an error state and the health indicator shows the API as down.
> - The layout works at both desktop and mobile widths.

Step 3 – DevOps & Infrastructure

3.1 Containerization

- Write a **production-ready Dockerfile** for the API.
- “Production-ready” means, for example: a pinned base image, a non-root user, a `.dockerignore`, and a clear start command.
- **Bonus:** use a multi-stage build to minimize the image size and keep development dependencies out of the final image.

3.2 Infrastructure as Code

- Provision your AWS hosting with **Terraform** or **AWS CDK**.
- **Default option:** an EC2 instance that runs your container, with the necessary security groups (allow SSH and inbound API traffic on your target port).
- **Alternative:** if you find a simpler way to host the backend and frontend on AWS (e.g., ECS on Fargate, App Runner, or S3 + CloudFront for the frontend), go for it, as long as it’s still defined in Terraform or CDK. Explain your choice in the README.
- Manual provisioning in the AWS Console is **not accepted**.

3.3 CI/CD pipeline

Create an automated workflow in **GitHub Actions** or **GitLab CI** that runs on every push to `main` and:

1. Installs dependencies and runs the unit tests. The pipeline stops if any test fails.
2. Builds the Docker image.
3. Deploys the updated container automatically to your EC2 instance (or the AWS service you chose), with no manual steps.

Store AWS credentials and SSH keys as CI secrets. If you push the image to a registry, which one is up to you (e.g., Amazon ECR, Docker Hub, or GitHub Container Registry).

> **Done when**
>
> - Running `terraform apply` or `cdk deploy` creates your environment from scratch.
> - A push to `main` that changes the API ends with the new version live, with no manual steps.
> - A push with a failing test does not deploy.

Step 4 – Documentation & System Architecture

Add a comprehensive `README.md` at the root of your repository that covers the following.

Setup and usage

- How to run the API and the frontend locally.
- How to run the tests and view the coverage report.
- How the infrastructure is provisioned and how deployment works.
- Any assumptions you made, and anything you didn't finish.

System Architecture section

Answer these three questions. Be specific to your design rather than generic.

Topic	Question
Scalability	If your API traffic suddenly increases by **10x tomorrow**, what specific changes would you make to this architecture to handle the load?
Observability	How would you set up production monitoring, error tracking, and log aggregation for this service?
AI utilization	Briefly mention where AI tools (Claude, Copilot, etc.) helped you, and describe one instance where you had to correct or refine the AI's output.

Submission

Send the following to your recruiting contact:

Deliverable	Details
Repository	Link to your public GitHub or GitLab repository with all code: API, frontend, Dockerfile, infrastructure code, and pipeline config.
Live URL	The public IP or DNS name of your running EC2 instance (or other AWS deployment), so we can call `GET /health` and open the frontend. Keep it running until your interview.
README / pull request	Instructions to run the project locally and execute the tests, plus deployment notes and the System Architecture section from Step 4.

Final checklist

- ☐ `GET /health`, `GET /tasks`, and `POST /tasks` work, and `POST /tasks` validates its input
- ☐ Custom request-logging middleware
- ☐ Jest backend coverage ≥ 80%, with data access mocked
- ☐ React app: task list, create form, loading/empty/error states, health indicator
- ☐ Production-ready Dockerfile (bonus: multi-stage build)
- ☐ AWS infrastructure in Terraform or CDK, with nothing created manually
- ☐ CI/CD on push to `main`: test → build → deploy
- ☐ README: local setup, tests, deployment, and System Architecture answers
- ☐ Repository link and live URL sent

What we'll evaluate

Area	What we look at
Functionality	The endpoints and UI behave as specified on your live deployment.
Code quality	Clear structure, readable code, correct async patterns, and sensible error handling.
Testing	Meaningful tests with mocked data access and at least 80% backend coverage.
Production readiness	Dockerfile quality, infrastructure as code, and a pipeline that tests and deploys automatically.
Architectural thinking	Specific, realistic answers on scalability and observability.
Ownership	Your ability to explain and defend your decisions.

The technical interview

Your submission is the basis for the technical interview. Expect to walk us through your code, infrastructure, pipeline, and deployment, and to explain the reasoning behind your choices, including the parts AI helped you write.

Use AI as much as you want. Just make sure you understand your code.